

Plan de Proyecto

Practica Final 2026 - Metodologia de la Programacion + Estructuras de Datos

Juego provisional: **Escape de la Mazmorra**

Grupo de 3 personas | Entrega: jueves 28 de mayo de 2026

Persona A	Estructuras de datos y matriz/grafó
Persona B	Logica del juego y reglas
Persona C	JavaFX, JSON y documentacion

Objetivo: entregar un proyecto pequeño pero sólido, usando estructuras propias, con una interfaz clara, pruebas, memoria, UML, JSON y video.

1. Que Tenemos Que Hacer

Tenemos que crear un juego por turnos en Java con interfaz grafica JavaFX. El jugador se mueve dentro de habitaciones representadas como matrices propias, y las habitaciones estan conectadas entre si formando un grafo propio. El objetivo es salir de la red de habitaciones antes de quedarse sin vida o sin turnos.

La practica no busca un videojuego enorme: busca demostrar diseno orientado a objetos, metodologia, estructuras de datos propias, persistencia en JSON, pruebas, UML, logs, gestion de errores y una interfaz clara.

Resumen del juego propuesto: Escape de la Mazmorra

El jugador despierta en una mazmorra formada por varias salas conectadas. Para escapar debe explorar habitaciones, esquivar trampas, recoger objetos, derrotar o evitar enemigos, encontrar llaves y llegar a una puerta final. Cada sala se muestra como una rejilla. Algunas puertas conectan con otras habitaciones y una de ellas lleva al exterior.

2. Requisitos Obligatorios En Orden De Trabajo

Orden	Bloque obligatorio	Resultado esperado
1	Entender el enunciado y decidir alcance minimo	Lista comun de requisitos y extras separados
2	Bocetar la interfaz antes de programar	Pantalla con mapa, estado, acciones, inventario y log
3	Definir requisitos, casos de uso e invariantes	Base de la memoria y metodologia
4	Disenar clases principales	Jugador, Habitacion, Celda, Objeto, Enemigo, Grafo, Partida
5	Reutilizar estructuras propias	Listas, colas, pilas, grafos y arboles binarios adaptados
6	Crear matriz propia sin arrays Java	Habitaciones con filas y celdas usando estructuras propias
7	Crear grafo propio de habitaciones	Mapa conectado con habitacion inicial y final
8	Implementar movimiento interno	Movimiento ortogonal, sin diagonales, con casillas alcanzables
9	Implementar turnos	Jugador primero, despues enemigos
10	Implementar objetos e inventario	Recoger, usar, equipar lo minimo necesario
11	Implementar combate	Formula del enunciado, vida nunca negativa
12	Implementar victoria y derrota	Salida final, sin vida o sin turnos
13	Cargar configuracion inicial desde JSON	Partida inicial definida en fichero
14	Guardar y cargar partida en JSON	Estado recuperable
15	Crear JavaFX basico	Matriz visible, acciones, estado, inventario y log
16	Añadir excepciones y log	Errores controlados y operaciones registradas
17	Preparar pruebas JUnit	Estructuras, logica y JSON no visual
18	Cerrar memoria, UML, diario IA y video	Entrega completa en GitHub, ZIP y Blackboard

3. Prioridades Del Proyecto

Prioridad	Elemento	Decision del grupo
P0	Estructuras propias	No usar ArrayList, HashMap, LinkedList, arrays Java ni equivalentes
P0	Grafo de habitaciones	Mapa principal del juego
P0	Matriz propia de habitaciones	Lista de filas + lista de celdas, sin Celda[][]
P0	Movimiento, turnos y combate	Primero que sea jugable, aunque sea simple
P1	JSON	Gson permitido; cargar configuracion y guardar/cargar partida
P1	JavaFX	Interfaz clara antes que bonita; GridPane recomendado
P1	Pruebas JUnit	Todas las clases no visuales importantes
P1	Memoria, UML y diario IA	Se trabajan durante el proyecto, no al final
P2	Extras	Solo despues de tener version funcional

4. Extras Recomendados Para Subir Nota

Extra	Dificultad	Por que conviene
Puertas con llave	Media	Encaja con objetos, inventario y puertas
Pociones fungibles	Baja	Facil de explicar y demostrar
Trampas de un solo uso	Baja	Aprovecha celdas interactivas y log
Objeto equipable simple	Media	Permite arma o escudo activo sin complicarlo demasiado
Pista de camino minimo	Media/Alta	Aprovecha grafo/BFS y cumple parte importante del enunciado
Arbol de acciones	Media	Permite justificar el uso de arbol binario
Contador por habitacion	Baja/Media	Extra sencillo si la logica de turnos esta clara
Asistente de consejos	Media	Alternativa al chatbot: mensajes segun estado del juego, sin IA real

Sobre el chatbot

No se recomienda como ampliacion principal porque puede consumir mucho tiempo y no esta directamente relacionado con la evaluacion. Mejor hacer un asistente de consejos: avisa de pocos turnos, enemigos cercanos, llaves pendientes o pista disponible. Es mas sencillo, demostrable y conectado con el juego.

5. Estructuras De Datos Que Vamos A Usar

Estructura propia	Uso en el juego	Responsable	Prioridad
Lista enlazada	Inventario, enemigos, objetos, habitaciones, log	Persona A	Imprescindible
Cola	Orden de turnos: jugador y enemigos	Persona A + B	Muy alta
Lista circular	Rotacion de entidades/turnos si se decide usar	Persona A	Alta
Grafo	Mapa de habitaciones conectadas	Persona A	Imprescindible
Matriz propia	Celdas de cada habitacion sin arrays Java	Persona A	Imprescindible
Arbol binario	Arbol de acciones o menu logico de acciones posibles	Persona A + B	Recomendable
Pila	Historial o deshacer accion, si sobra tiempo	Persona A	Opcional
BFS	Casillas alcanzables y caminos minimos sin coste	Persona A + B	Muy alta

Decision clave: matriz sin arrays

Cada habitacion se modelara como lista de filas, y cada fila como lista de celdas. Asi se cumple que la matriz usa estructuras propias. No se usara Celda[][] ni arrays para representar habitaciones.

6. Reparto General Entre Personas

Persona	Responsabilidad principal	Tambien ayuda en
Persona A	Estructuras propias, matriz, grafo, costes y pruebas de estructuras	BFS, JSON de configuracion, integracion
Persona B	Logica del juego: turnos, movimiento, combate, inventario, victoria/derrota	Pruebas de logica, conexion con JavaFX
Persona C	JavaFX, JSON, documentacion, memoria, diario IA, video	Bocetos, pruebas JSON, integracion final

Horas recomendadas: 8-10 horas semanales por persona. Persona C puede necesitar 9-12 horas algunas semanas porque JavaFX y JSON son los puntos mas flojos del grupo.

7. Calendario General Por Fases

Fase	Fechas limite	Objetivo	Entregable interno
Fase 1 - Diseno	2-8 mayo	Entender enunciado, alcance, bocetos, requisitos, casos de uso y modelo	Documento de decisiones y bocetos
Fase 2 - Base tecnica	9-15 mayo	Estructuras, matriz, grafo, modelo y primeras pruebas	Nucleo no visual preparado
Fase 3 - Juego funcional	16-22 mayo	Movimiento, turnos, combate, JSON y primera JavaFX	Primera partida jugable
Fase 4 - Cierre	23-28 mayo	Pruebas, bugs, memoria, UML, diario IA, video y entrega	Version final entregable

8. Fechas Limite Por Persona

Fecha	Persona A - Estructuras	Persona B - Logica	Persona C - JavaFX/JSON/Docs
3 mayo	Revisar estructuras reutilizables	Listar reglas principales del juego	Preparar documento compartido y portada de memoria
6 mayo	Proponer matriz propia y grafo	Casos de uso: mover, atacar, recoger, usar, cambiar sala	Bocetos de interfaz
8 mayo	Tabla estructura/uso/coste	Modelo de turnos y combate	Formato inicial de JSON
11 mayo	Listas, cola y matriz adaptadas	Jugador, enemigo, objeto y celda definidos a nivel logico	Proyecto JavaFX preparado o investigado
13 mayo	Grafo y BFS planteados	Movimiento interno y turnos basicos	Plantilla memoria + diario IA
15 mayo	Pruebas de estructuras	Combate e inventario basico	Primer JSON de configuracion
18 mayo	Ayuda en caminos y conexiones	Movimiento entre habitaciones y victoria/derrota	Carga JSON inicial
20 mayo	Revisar integracion de estructuras	Conectar logica con acciones	Interfaz: matriz + estado jugador
22 mayo	Corregir fallos de estructuras	Primera partida completa	Botones, inventario y log visible
24 mayo	Pruebas finales estructuras/grafos	Pruebas finales logica	Guardar/cargar partida
26 mayo	Justificacion de costes en memoria	Explicar reglas y pruebas en memoria	UML final, diario IA y critica del proyecto
27 mayo	Revision final y video	Revision final y video	ZIP, GitHub, video y comprobacion Blackboard

9. Checklist Persona A - Estructuras

Fecha	Checklist
3 mayo	☐ Revisar practicas anteriores y localizar listas, colas, pilas, grafos y arboles binarios
6 mayo	☐ Decidir como sera la matriz propia sin arrays Java
8 mayo	☐ Preparar tabla de estructuras: uso, motivo y coste
11 mayo	☐ Adaptar lista enlazada, cola y matriz propia al proyecto
13 mayo	☐ Adaptar grafo de habitaciones y plantear BFS
15 mayo	☐ Tener pruebas JUnit basicas de listas, cola, matriz y grafo
18 mayo	☐ Ayudar a calcular caminos entre habitaciones y casillas alcanzables
22 mayo	☐ Revisar que no se han usado colecciones prohibidas
24 mayo	☐ Cerrar pruebas de estructuras y grafo
26 mayo	☐ Escribir justificacion de estructuras y costes para la memoria
27 mayo	☐ Participar en video y revision final

10. Checklist Persona B - Logica Del Juego

Fecha	Checklist
3 mayo	☐ Definir reglas minimas: turnos, movimiento, acciones, combate y derrota/victoria
6 mayo	☐ Redactar casos de uso principales
8 mayo	☐ Cerrar modelo de turnos, movimiento y combate
11 mayo	☐ Definir jugador, enemigo, objeto, celda y habitacion a nivel logico
13 mayo	☐ Tener movimiento interno y turnos basicos
15 mayo	☐ Tener combate, inventario y objetos minimos
18 mayo	☐ Implementar movimiento entre habitaciones y salida final
20 mayo	☐ Conectar acciones del jugador con la interfaz
22 mayo	☐ Primera partida jugable completa
24 mayo	☐ Pruebas de movimiento, combate, inventario, victoria y derrota
26 mayo	☐ Ayudar en memoria: reglas, decisiones y pruebas
27 mayo	☐ Participar en video y revision final

11. Checklist Persona C - JavaFX, JSON Y Documentacion

Fecha	Checklist
3 mayo	[] Crear documento compartido de decisiones y estructura de memoria
6 mayo	[] Preparar bocetos de interfaz: mapa, estado, acciones, inventario y log
8 mayo	[] Definir JSON inicial y JSON de guardado
11 mayo	[] Investigar JavaFX basico: ventana, GridPane, botones, paneles
13 mayo	[] Mantener diario IA actualizado desde el primer uso
15 mayo	[] Preparar primer JSON de configuracion de ejemplo
18 mayo	[] Cargar configuracion inicial desde JSON con Gson
20 mayo	[] Mostrar matriz y estado del jugador en JavaFX
22 mayo	[] Mostrar acciones, inventario y log en JavaFX
24 mayo	[] Guardar/cargar partida y probar errores JSON
26 mayo	[] Cerrar UML, memoria, critica del proyecto y diario IA
27 mayo	[] Preparar ZIP, comprobar GitHub, grabar video y revisar Blackboard

12. Reuniones Y Forma De Trabajar

Momento	Que hacer
Cada 2 días	Reunion corta de 20-30 minutos: estado, bloqueos y siguiente tarea
Cada viernes	Integracion comun en GitHub y prueba de lo que haya
Cada cambio importante	Actualizar decisiones en la memoria o documento compartido
Desde el día 1	Registrar uso de IA: prompt, resultado, cambios y critica
Ultima semana	Nada de extras nuevos: solo bugs, pruebas, memoria y entrega

13. Riesgos Y Como Evitarlos

Riesgo	Impacto	Prevencion
Empezar por JavaFX	Mucho tiempo perdido si la logica cambia	Primero nucleo no visual
Usar arrays o colecciones prohibidas	Riesgo serio de suspender EEDD	Revisar imports y estructuras antes de entregar
JSON demasiado tarde	Guardar/cargar puede romper el modelo	Definir JSON el 8 mayo y probar el 18 mayo
Enemigos demasiado complejos	Consume tiempo y mete bugs	IA simple: acercarse y atacar
Extras antes del minimo	Proyecto incompleto	Extras solo desde el 22 mayo si todo funciona
UML desactualizado	Penaliza metodologia	Actualizar despues de la primera version funcional
Video al ultimo minuto	Problemas de tamano o subida	Grabar el 27 mayo como tarde
No todos entienden el sistema	Problemas en video/defensa	Revisiones conjuntas semanales

14. Checklist Final De Entrega

Categoria	Comprobacion
Juego	☐ Movimiento dentro de habitacion ☐ Movimiento entre habitaciones ☐ Turnos ☐ Combate
EEDD	☐ Grafo propio ☐ Matriz propia sin arrays ☐ Listas propias ☐ Cola propia ☐ Arbol justificado
JSON	☐ Configuracion inicial ☐ Guardar partida ☐ Cargar partida ☐ Errores controlados
Interfaz	☐ Matriz visible ☐ Estado jugador ☐ Inventario constante ☐ Acciones ☐ Log
Documentos	☐ Memoria PDF ☐ Critica del proyecto ☐ UML completo ☐ Bocetos ☐ Diario IA
Pruebas	☐ JUnit estructuras ☐ JUnit logica ☐ JUnit JSON ☐ Logs revisados
Entrega	☐ GitHub ☐ ZIP ☐ Video con todos ☐ Nombres en portada ☐ Entrega en ambas asignaturas

Regla de oro

Mejor un juego pequeno, claro, probado y bien documentado que un juego enorme, desorganizado y dificil de explicar. La prioridad es cumplir lo obligatorio con estructuras propias y dejar los extras para cuando la version funcional ya exista.

15. Calendario Final De Mayo 2026

Este calendario marca los objetivos que deberian estar cerrados al final de cada semana. Si una semana no llega a su objetivo, la siguiente debe empezar recortando extras, no retrasando lo obligatorio.

Lunes	Martes	Miercoles	Jueves	Viernes	Sabado	Domingo
				1 Previo	2 Inicio: leer PDF	3 Idea y alcance
4 Requisitos	5 Casos de uso	6 Bocetos	7 Modelo clases	8 FIN S1: diseno cerrado	9 GitHub	10 Estructuras
11 Matriz propia	12 Grafo	13 Turnos	14 Combate	15 FIN S2: nucleo no visual	16 Habitaciones	17 Victoria/derrota
18 JSON config	19 Guardar/cargar	20 JavaFX matriz	21 Acciones/log	22 FIN S3: partida jugable	23 Pruebas	24 Bugs y errores
25 UML final	26 Memoria + IA	27 Video + ZIP	28 ENTREGA Blackboard	29	30	31

Objetivo semanal	Debe estar terminado al cerrar la semana
Fin Semana 1 - 8 mayo	Enunciado entendido, idea provisional, alcance minimo, bocetos, requisitos, casos de uso, modelo inicial, estructuras decididas y JSON planteado.
Fin Semana 2 - 15 mayo	Estructuras propias adaptadas, matriz sin arrays, grafo, turnos basicos, movimiento, combate/inventario inicial y pruebas de estructuras.
Fin Semana 3 - 22 mayo	Primera version jugable: cambiar habitaciones, victoria/derrota, JSON basico, JavaFX con matriz, acciones, estado, inventario y log.
Fin Semana 4 - 28 mayo	Pruebas, correccion de bugs, excepciones, logs, memoria, UML, diario IA, critica del proyecto, video, ZIP, GitHub y entrega en ambas asignaturas.